High Throughput Cholesky Decomposition Based on FPGA

Jun Luo, Qijun Huang, Sheng Chang, Xiaoying Song, Yun Shang
Department of Electronic Science and Technology, School of Physics and Technology
Wuhan University
Wuhan, 430072, China

Abstract—**Cholesky decomposition has wide applications in solving many engineering and scientific problems. Acceleration is an important issue in many of these problems. In this paper, a hardware-based LL^T Cholesky decomposition featuring high throughput has been presented to solve wiener filtering based on the minimum square error criterion. To achieve the best efficiency, the hardware-based implementation has been realized by fixed-point multiple structures and various pipeline stages. Parallel properties have been exploited to improve the throughput. Results have shown that a significant speedup has been achieved compared to the software-based approach.**

Keywords-Wiener filtering; Cholesky decomposition; FPGA

I. INTRODUCTION

Solving a system of linear equation is a key problem in engineering and science. There are many approaches to solve linear system of equations. They can be generally classified into two categories by their nature. Direct methods include algorithms Gaussian elimination, Gaussian-Jordan elimination and Cholesky decomposition. Indirect approaches include Gaussian-Seidel and Newton-Raphson algorithms. The chosen of the method depends on the requirement of the specific application such as accuracy, structure and characteristic of the input matrix. Matrix factorization plays an important role in solving such equations. Cholesky decomposition is widely used in wireless communication, multiple spectral analysis and scientific computation for its fast execution time, easy adaption to parallel structures and low memory requirement compared to LU decomposition [1]. The Cholesky decomposition is based on the Cholesky Factorization Theorem [2].

Theorem: Given a symmetric positive definite matrix A, there exists a lower triangular matrix L such that $A=LL^T$.

The lower triangular matrix L is known as the Cholesky factor and LL^T is known as the Cholesky factorization of A. It is unique if the diagonal elements of L are restricted to positive. However, Cholesky decomposition is a time consuming process with a complexity of O ($N^3/3$). Row-oriented (left looking), column-oriented (right looking) and sub-matrix are three basic Cholesky algorithms which factor a positive symmetric definite matrix to the product of a lower triangular matrix and its transpose. There are two general ways of LL^T and LDL^T representing Chelosky decomposition, where L is a lower triangular matrix and D is a diagonal matrix.

So far, software- and hardware-based approaches have been developed to speed up the decomposition process. Yang [3] presented a Cholesky decomposition based on GPUs and FPGAs. He exploited the massively parallel computation for GPUs and his results show that the dedicated FPGA implementation has the highest efficiency. He applied the dedicated structure in compressed sensing reconstruction further [4]. However, it is normally expensive for a GPU and its performance is limited for a CPU [1] or a DSP approach [5, 6]. In [7], a software- and hardware co-design has been used in matrix manipulation with hardware taken as acceleration. This method implies a potential approach in solving Cholesky decomposition. FPGAs have played an important role in algorithm acceleration for their parallel properties. A fixed-point arithmetic unit operating in the rational fraction arithmetic was proposed in [8], which provided a configurable flexibility for FPGAs. But the improved precisions needed several iterations. A single triangular linear equation solver of Cholesky factorization was proposed in [9], which was mainly focused on the LDL^T representation. A rational fraction based arithmetic unit was adopted in [10] to solve $A=LL^T$ decomposition using FPGA. This method used a modified Cholesky algorithm to move the square root arithmetic to the last stage so as to reduce the hardware overhead of the processor array. In [11], a single precision and fused data path architecture was presented to implement Cholesky decomposition, but its throughput was limited due to the un-efficient manipulation for a whole matrix, which was needed to be loaded to the built-in memory. All in all, the methods descripted above were primarily concentrated on a specific architecture. Hence there are still large margins to be explored.

In order to solve wiener-hopf equations in wiener filtering, high throughput Chelosky decomposition with multiple parallel architectures has been presented and implemented by hardware in this paper. High throughput is achieved by exploiting the parallel properties of computing task. In the meantime, multiple pipeline technique has been investigated to achieve the best performance for Cholesky LL^T decomposition. The speedup between CPU and FPGA has been compared. Performance and complexity have been analyzed.

II. ALGORITHM OF CHOLESKY DECOMPOSITION

In Cholesky Factorization Theorem as introduced in section I, let element (l_{ij}) is the lower triangular of matrix L, the symmetric positive definite matrix A can be expressed by (1).

978-1-4799-2764-7/13/$31.00 ©2013 IEEE

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on September 16,2026 at 20:23:22 UTC from IEEE Xplore. Restrictions apply.

$$A=\begin{pmatrix} L_{11} & 0 & 0 \\ L_{21} & L_{22} & 0 \\ L_{31} & L_{32} & L_{33} \end{pmatrix}\begin{pmatrix} L_{11} & L_{21} & L_{31} \\ 0 & L_{22} & L_{32} \\ 0 & 0 & L_{33} \end{pmatrix} \quad (1)$$
$$=\begin{pmatrix} L_{11}^2 & L_{21}L_{11} & L_{31}L_{11} \\ L_{21}L_{11} & L_{21}^2+L_{22}^2 & L_{31}L_{21}+L_{32}L_{22} \\ L_{31}L_{11} & L_{31}L_{21}+L_{32}L_{22} & L_{31}^2+L_{32}^2+L_{33}^2 \end{pmatrix}$$

In (1), order three is used as the demonstration to obtain the following formula in solving elements (l_{ij}). As a result, elements (l_{ij}) of lower triangular matrix L can be determined by (2) and (3), in which a_{ij} is the element of matrix A.

$$l_{ij} = (a_{ij} - \sum_{k=1}^{j-1} l_{ik} l_{jk}) / l_{jj} \quad i > j \quad (2)$$

$$l_{ii} = \sqrt{a_{ii} - \sum_{k=1}^{i-1} l_{ik}^2} \quad (3)$$

Several Cholesky algorithms have been developed for matrix factorization. Right looking (column-oriented) algorithm is chosen is this work for its wide applications. The main operations in right looking algorithm are square root, division, multiplication and addition/ subtraction. To achieve a high utilization of memory resources, memory reusing is considered in the design structure. Half of one matrix storage memory is needed in decomposition. The pseudo-code of right looking algorithm is listed below:

```
for k=1:n
   a(k,k)=sqrt(a(k,k))
 for i=k+1:n
      a(i,k)=a(i,k)/a(k,k)
 endfor
 for j=k+1:n
      for i=j:n
        a(i,j)=a(i,j)-a(i,k)*a(j,k)
      endfor
 endfor
endfor
```

III. METHODOLOGY AND ARCHITECTURE

Wiener filtering is the optimal minimum square error (MSE) criterion for image restoration. In order to obtain wiener filtering coefficients, wiener-hopf equations are to be solved. After the auto- and cross-correlation matrixes are estimated, the key in solving wiener-hopf equations is the matrix factorization. Since the auto-correlation matrix is symmetric positive definite, Cholesky decomposition is applied in matrix factorization to achieve the best performance in solving wiener-hopf equations.

Traditionally, Cholesky factorization is implemented using float-point representation to achieve a high precision compared to fixed-point approach. However, the float-point method has a high complexity and high power consumption in hardware processes. To design a high throughput and low complexity structure for Cholesky decomposition, fixed-point representation is chosen in this paper. To achieve the best tradeoff between hardware complexity and image quality, the fixed-point bit width has been experimented according to peak signal to noise ratio (PSNR). Two window sizes (3×3 and 5×5) are tested in Fig. 1. It can be seen from Fig. 1 that the image quality (PSNR) is improved with the increased fixed-point bits in Cholesky decomposition and it can also be pointed out that the bit width should be more than 26 bits to get a PSNR improvement (anchor 38.928dB). Thus, 28 bits of fixed-point representation is chosen in this paper (each value is left shifted by 26).

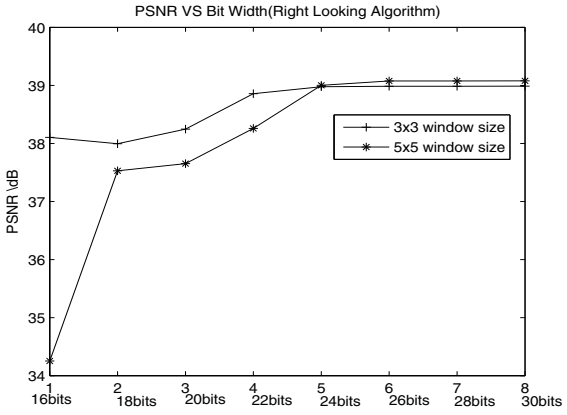


Figure 1. Bit width on the influence of PSNR

Efficient structure is a key for the performance of a Cholesky decomposition. Multiple architectures have been presented in this paper. There are two basic tasks in Cholesky factorization. Task *cdiv* is used to perform square root and column division while task *refresh* is an update task used for multiplication as they are shown in Fig. 2. To speed up the decomposition process, the parallel properties between task *cdiv* and task *refresh* are exploited. In Fig. 2, *cdiv* 2 is starting to operate before *refresh* 1 is completed (only after column 2 is refreshed) so that process time of factoring one matrix can be reduced. n is the matrix order and task i is focused on column i with its refreshing to column j ($j \in (i, n]$). The process chart of solving one matrix (n=5) is illustrated in Fig. 3. Half of the symmetric matrix data is needed to store in memory. The matrix data is decomposed column by column with the rest columns being refreshed by the decomposed column. With more columns to be decomposed, the process time of task *cdiv* and *refresh* is decreased accordingly.

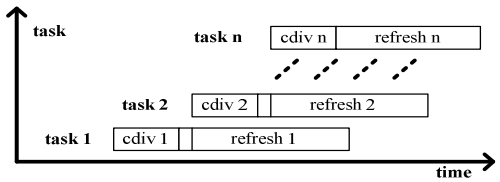


Figure 2. Task scheduling

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on September 16,2026 at 20:42:09 UTC from IEEE Xplore. Restrictions apply.

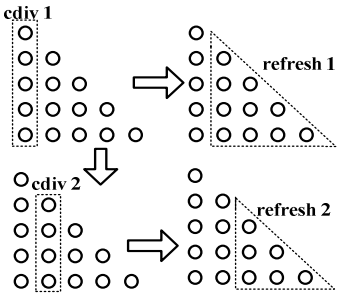


Figure 3. Process of Cholesky decomposition

The proposed structure is shown in Fig. 4, where column *i* and column *i+j* are on-chip memories used for storage. To get a fully utilization of memory resources, the updated data in Fig. 4 stores back to the same memory place before it is refreshed. There are *n* memory arrays. The maximum depth of each memory array is decreased column by column. In the *cdiv* process, first column data from matrix is squared (*sqrt* module) before divided (*divider array* module). In the refresh process, followed column data is updated. The tradeoff between complexity and performance is analyzed. To evaluate the structure on influence of performance for different matrix order, three kinds of structures have been presented in this paper. In the first structure (S1), only a divider and a multiplier are adopted in *divider* array and *mult* array modules respectively. This can be considered as the serial decomposition. Parallel multiplications with a divider are presented in structure two (S2). Finally, parallel of multiplications and divisions are proposed in the third architecture (S3). The parallel properties are increased from S1 to S3. Thus, the throughput of S1 to S3 will be increased theoretically. To evaluate the influence of proposed architectures on different matrix sizes, two kinds of matrix orders (9 and 25) have been experimented in this paper. The matrixes are dense positive symmetric with fixed-point (28 bits) width.

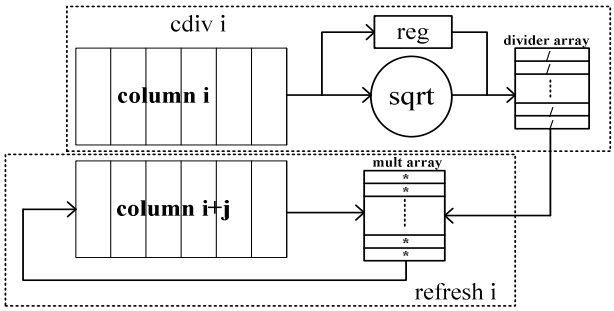


Figure 4. Proposed structure

IV. EXPERIMENTS AND DISCUSSION

Three structures with two kinds of matrix orders have been presented and synthesized by Quartus II 10.1 based on a Stratix II device EP2S130F1508C3. The square root, divider and multiplier are implemented by Intellectual Property (IP) core provided by Altera. Functional evaluation has been completed based on Modelsim SE 6.2b and Matlab 7.0. The verification of Cholesky decomposition with order 9 under S2 is illustrated in Fig. 5. Two order sizes under three structures with different pipeline stages have been functionally tested and verified. Simulation results have shown that the hardware based Cholesky decomposition can match that of software-based approach correctly.

```
# Data before cholesky decompostion:
#
#  67108864
#  66433283, 67108864
#  65618596, 66433283, 67108864
#  65897044, 65490337, 64852908, 67108864
#  65126820, 65897044, 65490337, 66433283, 67108864
#  64338576, 65126820, 65897044, 65618596, 66433283, 67108864
#  64512554, 64208594, 63784973, 65897044, 65490337, 64852908, 67108864
#  63787801, 64512554, 64208594, 65126820, 65897044, 65490337, 66433283, 67108864
#  63061444, 63787801, 64512554, 64338576, 65126820, 65897044, 65618596, 66433283, 67108864
#
# cholesky decomposition started at              1.090us
# Data after cholesky decompostion:
#
#  8192
#  8110, 1156
#  8010, 1274, 1151
#  8044,  219,  123, 1530
#  7950, 1231,  211, 1430,  549
#  7854, 1238, 1224, 1320,  590,  550
#  7875,  296,  286, 1602,  307,  -17, 1504
#  7787, 1176,  292, 1434,  784,  295, 1427,  457
#  7698, 1174, 1178, 1316,  641,  775, 1343,  502,  534
#
# cholesky decomposition ended at             10.110us
# Verification between FPGA and Matlab is successful!
#
VSIM 2> quit
G:\Hardware\cholesky\Right\tap9>
```
Figure 5. Functional verification of Cholesky decomposition under order 9

Different matrix sizes (order 9 and 25) with the three proposed structures have been synthesized and analyzed. The synthesized results with divider (20 stages of pipeline) and square root (10 stages of pipeline) for the three structures (S1, S2 and S3) are shown in Table I. The process time (*us*) of solving one matrix in Table I is based on the frequency of 100MHz. It can be seen from Table I that the process time in solving one matrix is decreased rapidly with more parallel properties exploited (from S1 to S3). Besides, more resources are consumed to improve performance (about 2× resources consumed to obtain 3× performance improvements under order 9). For most of the applications, a competitive option is S2

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on September 16,2026 at 20:42:09 UTC from IEEE Xplore. Restrictions apply.

considering complexity and performance. Because the resource cost in S2 is equivalent to that in S1 except the additional multipliers while the performance improved a lot (the F_{max} improved in both order 9 and 25). More improvements can be obtained in higher orders (order 25). Throughput is taken as the performance evaluation criterion. It can be derived by (4), where N_T is the number of samples, F_{max} is the maximum frequency and T_{total} is the decomposition time of each matrix.

$$Throughput = N_T \times \frac{F_{\max}}{T_{total}} \quad (4)$$

To analyze different pipeline stages on the influence of performance, synthesized results of three kinds of pipeline stages in S2 for order 9 and 25 are summarized in Table II. It can be found from our experiment that the throughput can be improved with the decreased pipeline stages when the numbers of dividers are small. When the numbers of dividers are increased, more throughputs will be achieved when the stages of pipeline are much more. This is mainly because the maximum frequency decreases rapidly when more dividers are adopted. It can also be found out that the highest throughput can be achieved when there are 10 stages of pipeline for divider and 5 stages of pipeline for square root in Table II. This can be explained by that fewer pipelines can decrease T_{total} rapidly and the decreasing rate of T_{total} is faster than F_{max}. (Fewer pipelines will decrease the maximum frequency theoretically).

TABLE I. SYNTHESIZED RESULT OF DIFFERENT STRUCTURES

Order	**9**			**25**		
Structure	***S1***	***S2***	***S3***	***S1***	***S2***	***S3***
ALUTs	4543	4268	9082	29684	28915	45322
Registers	4008	4303	6188	14626	15801	22358
Memory Bits	322	322	3544	322	322	10632
DSP 9bits	2	16	16	2	48	48
Solve one matrix (us)	10.1	9.02	3.62	134.03	65.03	25.03
F_{max} (MHz)	148.92	168.92	159.67	118.39	124.66	133.94

TABLE II. PERFORMANCE OF DIFFERENT PIPELINES FOR DIVIDER AND SQUARE ROOT UNDER S2

Order	**9**			**25**		
Pipelines (divider, square root)	***(20,10)***	***(10,10)***	***(10,5)***	***(20,10)***	***(10,10)***	***(10,5)***
ALUTs	4268	4200	4123	28915	29573	28376
Registers	4303	4014	3870	15801	15516	15372
Memory bits	322	88	88	322	88	88
DSP 9bits	16	16	16	48	48	48
Solve one matrix (us)	9.02	5.42	4.97	65.03	35.03	33.78
F_{max} (MHz)	168.92	125.88	124.8	124.66	117.55	119.6
Throughput (Msps)	8.427	10.451	11.299	6.23	10.906	11.507

To evaluate the efficiency of FPGA implementation for Cholesky factorization, the throughput of software- and hardware- based Cholesky decomposition has been compared in Table III. The software is run on a personal computer (PC) with an Intel Pentium 4 processor (3GHz maximum frequency) and 2Gbits random access memory. Chol_u is the implemented function by Matlab and chol is the provided function by LAPCK (Linear Algebra Package). Run time of solving one matrix is measured in Matlab by averaging 10,000 times for the sake of reducing accidental error. It can be found that the hardware-based decomposition can achieve a speedup over software-based (LAPACK) approach. There is speedup of 3× (S2) and 6× (S3) in order 9 and speedup of 1× (S2) and 2× (S3) in order 25 approximately. More speedup is acquired compared to the implemented function chol_u.

TABLE III. THROUGHPUT (MSPS) COMPARISON BETWEEN CPU AND FPGA

Order	**Function**	**CPU**	**FPGA**		**Speed up**	
			S2	***S3***	***S2***	***S3***
9	chol_u	0.84	11.29	19.84	13.3	23.6
	chol	3.2			3.5	6.2
25	chol_u	1.16	11.5	17.39	9.9	14.9
	chol	8.52			1.3	2.0

An indirect run time comparison of solving one matrix under order 25 has been shown in Table IV. It can be seen that a speedup of 1232× is achieved for the proposed S1 structure compared to [1] with two processors in the worst case (134.03us×100MHz/33.33MHz=402.13us). More speedup can be obtained under the structure of S2 and S3, respectively.

TABLE IV. RUN TIME OF SOLVING ONE DENSE MATRIX UNDER ORDER 25

In [1] (based on Nios processor with 33.33MHz)		***One processor***	***Two processors***
		953.72ms	495.45ms
Proposed(based on FPGA with 100MHz)	S1	134.03us	
	S2	33.78us	
	S3	25.03us	

The number of clock cycles per decomposition compared to [5] is summarized in Table V. The pipeline stages of divider and square root are listed by the first and second number in the bracket. Although the matrix order is 8 in [5], it can be indicated that an improvement is gained in the proposed structure (more cycles are needed when the order is higher theoretically). It can be found from Table V that approximately 62% of fewer cycles are achieved in the proposed S3. Thus, high throughput Cholesky decomposition is achieved by exploiting parallel task scheduling and parallel properties of data dependency in structures.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on September 16,2026 at 20:42:09 UTC from IEEE Xplore. Restrictions apply.

TABLE V. NUMBER OF CLOCK CYCLES PER DECOMPOSITION

In [5](order 8)	Proposed (order 9)		
	S1(20, 10)	*S2(10, 5)*	*S3(20,10)*
961	1010	479	362

V. CONCLUSION

Solving linear equation is critical in MSE-based wiener filtering. Cholesky decomposition plays an important role in matrix factorization. Multiple structures with different pipeline stages for Cholesky decomposition have been investigated in this paper. Our results show that decreasing the pipeline stages will improve the throughput when the numbers of dividers are small. High throughput is achieved by exploiting the parallel properties of executing tasks and architectures. Future work includes implementing LDL^T decomposition and comparing CPU performance.

ACKNOWLEDGMENT

The work is supported by the Fundamental Research Funds for the Central Universities (2012202020204), and by Natural Science Foundation of Hubei Provincial, China (2011CDB272).

REFERENCES

[1] S. G. Haridas and S. G. Ziavras, "FPGA implementation of a Cholesky algorithm for a shared-memory multiprocessor architecture," Parallel Algorithms and Applications, vol. 19, pp. 211-226, December 2004.

[2] N. J. Higham, "Cholesky factorization," Wiley Interdisciplinary Reviews: Computational Statistics, 2009, pp. 251-254.

[3] D. Yang, J. Sun, J. Lee, G. Liang, D. D. Jenkins, G. D. Peterson and H. Li, "Performance comparison of Cholesky decomposition on GPUs and FPGAs," Proceedings of the Symposium Application Accelerators in High Performance Computing, 2010.

[4] D. Yang, G. D. Peterson and H. Li, "Compressed sensing and Cholesky decomposition on FPGAs and GPUs," Parallel Computing, vol. 38, no. 8, pp. 421-437, August 2012.

[5] P. Salmela, A. Happonen, T. Jarvinen, A. Burian, and J. Takala, "DSP implementation of Cholesky decomposition," SympoTIC'06 - Joint 1st Workshop on Sensor Networks and Symposium on Trends in Communications, pp. 6-9, June 2006.

[6] R. Maoudj, L. Fety and C. Alexandre, "Performance analysis of modified gram-schmidt Cholesky implementation on 16 bits-DSP-chip," International Journal of Computing and Digital Systems, vol. 2, no. 1, pp. 21-27, 2013.

[7] N. Dave, K. Fleming, M. King, M. Pellauer and M. Vijayaraghavan, "Hardware acceleration of matrix multiplication on a Xilinx FPGA," 5th IEEE/ACM International Conference on Formal Methods and Models for Codesign, pp. 97-100, 2007.

[8] O. Maslennikow, V. Lepekha, A. Sergiyenko, A. Tomas and R. Wyrzykowski, "Parallel implementation of Cholesky LL^T-Algorithm in FPGA-Based processor," Parallel Processing and Applied Mathematics, Springer Berlin Heidelberg, pp. 137-147, 2008.

[9] D. Yang, G. D. Peterson, H. Li, and J. Sun, "An FPGA implementation for solving least square problem," IEEE Symposium on Field Programmable Custom Computing Machines, FCCM, pp. 303-306, April 2009.

[10] O. Maslennikow, P. Ratuszniak, and A. Sergyienko, "Implementation of cholesky LLT-decomposition algorithm in FPGA-based rational fraction parallel processor," 14th International Conference "Mixed Design of Integrated Circuits and Systems", pp. 287-292, June 2007.

[11] S. S. Demirsoy and M. Langhammer, "Cholesky decomposition using fused datapath synthesis," 7th ACM SIGDA International Symposium on Field- Programmable Gate Arrays, pp. 241-244, February 2009.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on September 16,2026 at 20:42:09 UTC from IEEE Xplore. Restrictions apply.